NukMiris v0.8.1 — Independent Greenfield Architecture Challenge

Deliverable 4 of 5 — Reliability and Security Challenge

Independent architect submission · 26 October 2026 · I attack my own design here. Sections (§) refer to Deliverable 2, ADR-n to Deliverable 3, F-n, T-n and S-n to Deliverable 1.

For each scenario: **what happens**, **why it is hard**, **how the architecture responds**, **what remains**, and **how I would prove it**. “Remains” is deliberately candid.

1. Duplicate WhatsApp webhooks

What happens. Meta retries because our acknowledgement was slow; two copies reach two `web` instances at once; a copy arrives days later; the same ID arrives with different content; status notifications repeat.

Why it is hard. Duplicates can be concurrent, late, or subtly different, and the cost of an error is a double booking or a second reply.

Response.

1. The provider event key is unique. Concurrent inserts use `ON CONFLICT DO NOTHING`; the loser waits for the winner’s commit, sees the conflict, does nothing, and still returns 200 only after durable acceptance exists.
2. On conflict a SHA-256 content hash is compared. A mismatch becomes an `integrity_conflicts` row and an alert; the original is never substituted (I04).
3. Sequence numbers are allocated only for new events, so duplicates cannot reorder a conversation.
4. Processing is idempotent because effects and the `processed` mark share one transaction (§5.3).
5. A repeated customer tap resolves to the same operation through the unique source offer (I02).
6. Status duplicates collapse on message ID, status and provider timestamp.

What remains. Event keys must outlive any retention purge of payloads, longer than the provider’s retry horizon **[verify S-7]**. Message types whose identifiers differ from my assumption would weaken the key (S-7).

Proof. Replay every recorded payload N times, concurrently, across instances: exactly one processing, one reply, one operation. Property: dedupe holds after payload purge.

2. Concurrent bookings

What happens. Two customers select the same slot; one customer double-taps; two slots partially overlap; a booking races a reschedule; an owner creates a manual event at the same moment.

Response.

1. The reservation exclusion constraint serializes all NukMiris writers (§8.2). The loser receives a definite local rejection and fresh options.
2. Partial overlap, buffer and breaks are enforced by the same range (half-open, buffer included).
3. A double-tap hits the unique source offer and returns the recorded status.
4. A reschedule and a booking compete under one rule; a move may overlap only its own appointment key.
5. Before every insert a fresh uncached Google read catches events NukMiris did not create.
6. After the insert an overlap audit flags any non-NukMiris overlap to the review queue.

What remains. The window between the fresh read and the Google insert (hundreds of milliseconds) cannot exclude a manual or third-party writer. That is U-EXTERNAL: detected, not prevented. The owner must decide the commercial policy.

Proof. 50 concurrent bookers on one slot yield one winner and no committed overlap (sentinel 1). A simulator injects a foreign event between read and insert and asserts the audit flags it.

3. Lost Google Calendar responses

Operation	Ambiguity	How it is resolved	What the design never does
Insert	Event may exist	Read by the pre-allocated ID. Found with matching `nm_op`, times and ownership: applied. Absent after the settle window: recheck availability, then the *same-ID* insert, which either creates it once or returns a conflict that is then read and verified (ADR-006, ADR-009)	Insert with a new ID
Delete	Event may be gone	Read. Cancelled, or not found or gone on an event that was ours: applied	Report success without evidence
Move	Patch may have applied	Read. `nm_chg` equals this operation and times equal the target: applied. Original times and a different `nm_chg`: not applied, only after the settle window	Re-patch before the window passes
Attendance	Title may carry the suffix	Read for `nm_att` and the suffix once	Patch twice; acknowledge an unknown result
Read during reconciliation	The read itself fails	Stay uncertain; retry the read with backoff	Treat a failed read as “not found”
Malformed or truncated body	Cannot tell	Uncertain	Parse leniently into success

What remains. This depends on S-1 and S-2, and on how quickly a successful insert becomes readable. The settle window (10 minutes by default) is a guess until measured. Until S-1 is confirmed, the fallback is operator resolution.

Proof. The Calendar simulator offers *fail before commit* and *succeed, lose the response* faults for every operation; every run ends with one event or none, never two, and the customer never sees success for an unverified result.

4. Process crashes

Process and moment	State left behind	Recovery
`web` after commit, before returning 200	Event stored	Provider retries; dedupe makes it a no-op
`web` before commit	Nothing stored	Provider retries; stored on the retry
Mailbox mid-event	Event unprocessed, lease expires	Another worker claims and reprocesses; no external effect occurred inside the path (§5.3)
Executor, any point in §8.4	As tabulated there	Marker distinguishes never-sent from possibly-sent
Sender after `submitting` marker	Attempt `submitting`, lease expires	Becomes `uncertain`, never resent (§10)
Scheduler mid-batch	Some occurrences claimed	Leases expire; each occurrence is independent
Database failover	In-flight transactions roll back	Workers reconnect; every operation above is restartable

What remains. A crash between the submission marker and the socket write is classed uncertain although nothing left the machine. That is a deliberate trade: a rare missing message instead of a possible duplicate.

Proof. The crash-point harness kills at every named point, restarts on the same database, and asserts the outcome table plus all sentinels (I12).

5. Interrupted reminders

Interruption	Result
Crash before the attempt marker	Occurrence still pending; sent normally if within its allowance
Crash after marker, before the send	`uncertain`; not resent
Crash after the send, before the outcome is recorded	`uncertain`; resolved by receipt; not resent
Downtime longer than 15 minutes	Occurrence past `expires_at` is `skipped(late)`; never replayed on restart
Two workers claim the same occurrence	Lease and fence plus the unique occurrence key prevent a second attempt
Appointment moved or cancelled between claim and send	The fresh Calendar read in preflight cancels the occurrence
Permission withdrawn between enrollment and send	Preflight checks evidence at send time
Template paused in the meantime	Preflight blocks fail-closed; the occurrence is skipped when its allowance ends
Delivered receipt for the parent arrives after the follow-up is due	The occurrence stays pending until the allowance ends, so a late receipt can still satisfy R04
Clock skew between hosts	Only the database clock decides “due” (§4.3)

What remains. An accepted reminder is not proof of arrival; template TTL caps how late a delivery can be, but a message can still reach the device after the customer has acted. Device-arrival timing is outside our control (R03).

Proof. Virtual-clock tests at exactly 169 h, 168 h, 24 h, 22 h and 2 h; kill-and-restart at each interruption row; sentinels 7 and 8.

6. Out-of-order WhatsApp delivery receipts

What happens. `read` before `delivered`; `failed` after `delivered`; `sent` after `read`; duplicates; a receipt that arrives before our send call has even returned; a receipt for a message we never recorded.

Response.

1. The merge takes the maximum rank, with failure unable to lower `delivered` or `read`, so every arrival order converges (§10.3).
2. A receipt that arrives before the attempt row holds a provider message ID is correlated by the opaque attempt ID **[verify S-6]**. If correlation is not yet possible it is stored as an orphan, and the reconciler re-examines orphans against new attempts.
3. Correlation checks business, sending channel, recipient `wa_id` and message ID. A receipt that matches nothing exactly is an orphan with no effect, so it can never resolve another customer's uncertain send.
4. Receipt time is stored apart from the customer's device or click time (O01).

What remains. If the callback echo does not exist (S-6), some uncertain sends can only be resolved by an operator.

Proof. Property test: every permutation of a receipt multiset yields the same effective state. Simulator reorders and duplicates receipts across customers; assert zero cross-attempt resolution.

7. Cross-business access attempts

Attempt	Control	Test
Customer replays or guesses another business's button	128-bit opaque tokens validated against business and conversation	Token forging suite
Same phone contacts two businesses	Scoped customer rows; separate conversations	Two-tenant fixtures in every suite
Payload claims another phone-number identifier	Signature authenticates the sender; the channel table, not the payload's business, decides the tenant	Mutation fuzz

Attempt	Control	Test
Developer forgets a tenant filter	Forced RLS returns no rows when the setting is absent or different	Generated leakage test across all tables and repository functions
Worker claims across tenants	Narrow `SECURITY DEFINER` functions return only identifiers; processing reopens a scoped transaction	Review plus role-privilege test
Two businesses map one Calendar	Unique Calendar-to-business mapping (F-04)	Constraint test
An event from another business appears in a lookup	`nm_biz` predicate on every read	Simulator seeded with foreign events
Credential ciphertext copied between tenants	Authenticated data binds business and provider	Decrypt-with-wrong-AAD test
A receipt resolves someone else's send	Exact correlation or orphan	Cross-customer receipt test
Channel reassigned to another business	Operator action; audited; sessions and offers invalidated	Integration test
Operator targets the wrong business	Explicit business argument, echo-and-confirm for mutations, audit	CLI test
Customer data in telemetry	Redaction; masked identifiers	Log-redaction tests

What remains. The shared database is a shared blast radius: a bug in a `SECURITY DEFINER` function or a misconfigured role can still cross tenants. That is why role privileges are tested, not assumed.

8. Expired appointment selections

What happens. A customer taps a time after 15 minutes; taps an older superseded menu; taps after sending Menu; replays an old reminder button after the appointment began or the schedule changed; answers a name prompt that has expired.

Response.

1. Every action is validated against its offer set's `expires_at` and supersession on the database clock. A failure moves the conversation to ExpiredView with recovery guidance and **no appointment or customer effect** (I03).
2. An expired name prompt no longer protects input: the text is interpreted as an ordinary message in Main and **nothing is registered**.
3. Renewed views invalidate earlier offers, so an older menu's buttons stop working.
4. Reminder buttons are bound to occurrence and schedule identity and stay valid until appointment start; a changed schedule invalidates them (R07, R12).
5. A durable operation action is not a browsing offer, so replacing a view cannot orphan a recorded operation.

What remains. The customer experiences a dead button. The guidance text and a one-tap path to restart are the mitigation.

Proof. Virtual-clock tests at 14:59, 15:00 and 15:01; replay of every action type after supersession; assert no customer or operation row is created.

9. Credential failure or rotation

Event	Effect	Handling
Google consent revoked or refresh returns `invalid_grant`	The business cannot read or write its Calendar	`calendar_auth_failed`; availability is `Unavailable`; unsubmitted operations end as definite rejections; uncertain ones wait for restoration and are then reconciled; page
Failed new consent	Risk of losing the working grant	The working credential is replaced only after scopes and Calendar access are verified (I11)
Meta token expired mid-send	Provider rejects	Definitely rejected (credential); sender paused for that business; the retry after rotation is safe
Meta token rotation	Overlap needed	`next` version proven by a read-only probe, atomic switch, old version retired after grace

Event	Effect	Handling
App secret rotation	Signatures stop verifying	Two secrets accepted during overlap; a spike in signature failures alerts
Wrong secret deployed	Every webhook returns 401 and Meta retries	Alert within minutes; stored retries drain on recovery **[verify S-7]**
KMS unavailable	Cannot decrypt new tokens	In-memory token cache bridges short outages; page on sustained failure
Credential suspected stolen	Attacker may call provider APIs	Revoke at the provider, rotate, review the audit trail and provider logs

What remains. If a customer's Google authorization is revoked while an operation is uncertain, that operation cannot be proven until the customer re-consents. It stays in review with its reservation held.

10. Database restore after external side effects

Setup. The database is restored to time T0. Events after T0 did happen in the world: Google events were created, moved or cancelled; messages were delivered; customers pressed STOP.

Reality after T0	Restored database believes	Resolution
A booking exists in Google	No operation, no reservation	Calendar reconciliation by `nm_biz` and `updatedMin` finds an orphan; adoption is a reviewed operator action. Until then Google's busy time still blocks the slot.
A cancellation happened in Google	Appointment active	Sync sees the cancelled event; occurrences are invalidated before any send

Reality after T0	Restored database believes	Resolution
A reply was sent	Event unprocessed	Reprocessing may produce a second ordinary reply (accepted, bounded by the 24 h window)
A reminder was sent	Occurrence pending	Quarantine blocks sending; occurrences in the gap are marked uncertain; older ones are skipped by the 15-minute rule; archived receipts resolve some
A customer sent STOP	Evidence missing	The write-once ingress archive is replayed through the normal inbox, recovering the withdrawal **before** quarantine ends
An operator recorded consent evidence	Evidence missing	See the gap found below
A customer was mid-flow	State lost	Conversation resets; expired-view guidance on the next action
A credential was rotated	Old ciphertext restored	Keys and secrets follow their own recovery; re-probe every credential before leaving quarantine

A gap this scenario exposed. Operator mutations, such as consent evidence, configuration changes and credential rotations, live only in the database and would be lost after T0. I therefore add: **every operator mutation is also appended to the write-once archive as a command journal**, replayable during restore. (I am adding this to §16 of Deliverable 2.)

What remains. Messages sent after the last archived copy and the restore point cannot be known with certainty; the response is to hold, not resend. Loss of customer messages after the last archive export is bounded by export latency, which is why T-4 includes it.

Proof. An automated drill: run the canary workload, snapshot at T0, continue the workload, destroy the database, restore, run the §16.3 script, and assert: zero duplicate reminders, zero lost STOPs, all orphans surfaced, sentinels clean.

11. Further failures worth naming

Failure	Response	What remains
Database failover in the middle of a transaction	Transaction rolls back; every step is restartable; leases expire	A short stall (T-4 sets 15 minutes for failover)
Poison event that always fails processing	Parked after three attempts, review item, conversation continues	One customer message ignored until they resend
Manual external writer overlaps a booking	Detected by the overlap audit; review item; never auto-cancelled	A real double booking may exist until a human resolves it (U-EXTERNAL)
Template paused or rejected while live	Status notification blocks reminders fail-closed; page	Reminders stop until a template is re-approved
Government changes a time-zone rule	tzdata refreshed through the image pipeline	Events already written to Google follow Google's own zone data
Webhook flood	HMAC check before any database work; size limit; edge rate limits	Valid-signature floods from the provider itself are throttled only by capacity
Operator retries an uncertain message and duplicates it	Explicit reason, duplication warning, audit, linked attempt (U-RETRY)	The duplicate still happens if the operator chooses it
Customer in human mode is never answered	Handoff queue, oldest-unattended metric (F-11)	There is still no human process behind it

12. Where my own design is weakest

1. **Provider assumptions (S-1, S-6).** Two central mechanisms, idempotent insert and callback correlation, rest on behaviours I could not verify. If both fail, recovery depends far more on operators.
2. **Operational discipline.** The design is only as good as its drills, alerts and runbooks. A team that skips the restore drill has a plan, not a capability.

3. **Guessed timers.** Settle window, receipt-wait and polling intervals are reasoned defaults. Real provider latency may require retuning.
4. **Simulator fidelity.** My simulators encode my understanding. Drift jobs reduce this risk but cannot remove it.
5. **Single primary database.** A regional or provider-level failure means downtime up to the restore objective.
6. **Tenant isolation depends on discipline at the edges.** RLS and composite keys are strong, but `SECURITY DEFINER` functions and break-glass roles are privileged code paths that deserve review on every change.